

OOPS/ Inheritance

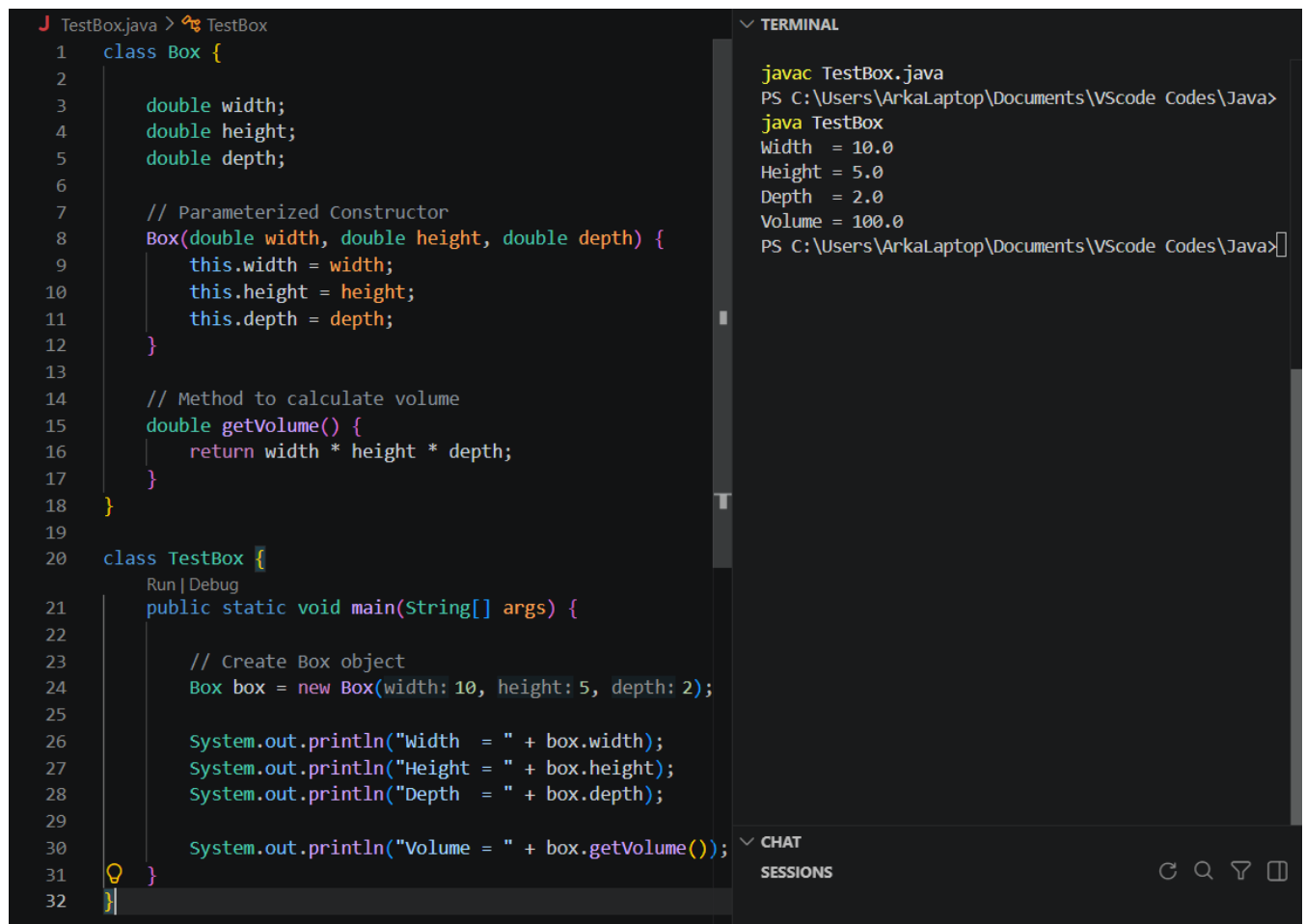
Classes & Objects

Q35. Create a class Box that uses a **parameterized constructor** to initialize the dimensions of a box.

The dimensions of the box are: Width, Height, Depth

The class should have a method that returns the **volume** of the box.

Create an object of the Box class and test the functionalities.



```
J TestBox.java > TestBox
1  class Box {
2
3      double width;
4      double height;
5      double depth;
6
7      // Parameterized Constructor
8      Box(double width, double height, double depth) {
9          this.width = width;
10         this.height = height;
11         this.depth = depth;
12     }
13
14     // Method to calculate volume
15     double getVolume() {
16         return width * height * depth;
17     }
18 }
19
20 class TestBox {
21     Run | Debug
22     public static void main(String[] args) {
23
24         // Create Box object
25         Box box = new Box(width: 10, height: 5, depth: 2);
26
27         System.out.println("Width = " + box.width);
28         System.out.println("Height = " + box.height);
29         System.out.println("Depth = " + box.depth);
30
31         System.out.println("Volume = " + box.getVolume());
32     }
33 }
```

TERMINAL

```
javac TestBox.java
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java>
java TestBox
Width = 10.0
Height = 5.0
Depth = 2.0
Volume = 100.0
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java>
```

CHAT SESSIONS

Q36. Create a new class called **Calculator** with the following methods:

1. A **static method** called `powerInt(int num1, int num2)`.
 - This method should return **num1 raised to the power num2**.
2. A **static method** called `powerDouble(double num1, int num2)`.
 - This method should return **num1 raised to the power num2**.
3. Invoke both the methods and test the functionalities.

Hint: Use `Math.pow(double, double)` to calculate the power.

```
J TestCalculator.java > TestCalculator
1  class Calculator {
2
3      // Static method for integer values
4      static int powerInt(int num1, int num2) {
5          return (int) Math.pow(num1, num2);
6      }
7
8      // Static method for double values
9      static double powerDouble(double num1, int num2) {
10         return Math.pow(num1, num2);
11     }
12 }
13
14 class TestCalculator {
15     Run | Debug
16     public static void main(String[] args) {
17
18         System.out.println("powerInt(2, 3) = " +
19             Calculator.powerInt(2, 3));
20
21         System.out.println("powerDouble(2.5, 3) = " +
22             Calculator.powerDouble(2.5, 3));
23     }
24 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java> javac TestCalculator.java
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java> java TestCalculator
powerInt(2, 3) = 8
powerDouble(2.5, 3) = 15.625
```

Encapsulation & Abstraction

Q37. A) Create a class **Author** with the following information: **Member Variables:** name (String), email (String), gender (char), Use a **parameterized constructor** to initialize the variables.

B) Create a class **Book** with the following information: **Member Variables:** name (String), author (Author object), price (double), qtyInStock (int)

Assumption: Each book is written by exactly one author.

- Use a **parameterized constructor** to initialize the variables.
- Create **getters and setters** for all member variables.

In the main() method:

- Create a Book object.
- Print all the details of the book (including the author details).

Code:

```
class Author {  
    private String name;  
    private String email;  
    private char gender;  
  
    // Parameterized Constructor  
    Author(String name, String email, char gender) {  
        this.name = name;  
        this.email = email;  
        this.gender = gender;  
    }  
  
    // Getters
```

```
public String getName() {  
    return name;  
}
```

```
public String getEmail() {  
    return email;  
}
```

```
public char getGender() {  
    return gender;  
}
```

```
// Setters
```

```
public void setName(String name) {  
    this.name = name;  
}
```

```
public void setEmail(String email) {  
    this.email = email;  
}
```

```
public void setGender(char gender) {  
    this.gender = gender;  
}  
}
```

```
class Book {
```

```
    private String name;  
    private Author author;
```

```
private double price;

private int qtyInStock;


// Parameterized Constructor
Book(String name, Author author, double price, int qtyInStock) {

    this.name = name;

    this.author = author;

    this.price = price;

    this.qtyInStock = qtyInStock;

}


// Getters

public String getName() {

    return name;

}


public Author getAuthor() {

    return author;

}


public double getPrice() {

    return price;

}


public int getQtyInStock() {

    return qtyInStock;

}


// Setters

public void setName(String name) {
```

```
        this.name = name;
    }

    public void setAuthor(Author author) {
        this.author = author;
    }

    public void setPrice(double price) {
        this.price = price;
    }

    public void setQtyInStock(int qtyInStock) {
        this.qtyInStock = qtyInStock;
    }
}

class TestBook {

    public static void main(String[] args) {

        // Create Author object
        Author author = new Author(
            "J.K. Rowling",
            "jkrowling@gmail.com",
            'F');

        // Create Book object
        Book book = new Book(
            "Harry Potter",
            author,
```

```
        599.50,  
        25);  
  
    // Print Book Details  
    System.out.println("Book Name : " + book.getName());  
    System.out.println("Price : " + book.getPrice());  
    System.out.println("Quantity : " + book.getQtyInStock());  
  
    // Print Author Details  
    System.out.println("Author Name : " + book.getAuthor().getName());  
    System.out.println("Author Email : " + book.getAuthor().getEmail());  
    System.out.println("Author Gender : " + book.getAuthor().getGender());  
    }  
}
```

Output:

```
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java> javac TestBook.java  
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java> java TestBook  
Book Name : Harry Potter  
Price : 599.5  
Quantity : 25  
Author Name : J.K. Rowling  
Author Email : jkrowling@gmail.com  
Author Gender : F
```

Inheritance

Q38. Create a class named **Animal** which includes the methods: eat(), sleep()

Create a child class of Animal named **Bird** and **override** the parent class methods.

Add a new method named: fly()

In the main() method:

- Create an object of the Animal class and invoke the eat() and sleep() methods.
- Create an object of the Bird class and invoke the eat(), sleep(), and fly() methods.

Code:

```
class Animal {  
    void eat() {  
        System.out.println("Animal is eating.");  
    }  
  
    void sleep() {  
        System.out.println("Animal is sleeping.");  
    }  
}
```

```
class Bird extends Animal {  
  
    // Overriding eat()  
    @Override  
    void eat() {  
        System.out.println("Bird is eating.");  
    }  
}
```



```
// Overriding sleep()
@Override
void sleep() {
    System.out.println("Bird is sleeping.");
}

// New method
void fly() {
    System.out.println("Bird is flying.");
}
}

class TestAnimal {

    public static void main(String[] args) {

        // Animal object
        Animal animal = new Animal();

        animal.eat();
        animal.sleep();

        System.out.println();

        // Bird object
        Bird bird = new Bird();

        bird.eat();
        bird.sleep();
    }
}
```

```
        bird.fly();  
    }  
}
```

Output:

```
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java> javac TestAnimal.java  
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java> java TestAnimal  
Animal is eating.  
Animal is sleeping.  
  
Bird is eating.  
Bird is sleeping.  
Bird is flying.
```

Q39. Create a class called **Person** with a member variable **name**. Save it in a file called **Person.java**.

Create a class called **Employee** that inherits the **Person** class. The Employee class should contain the following data members:

- annualSalary (double)
- yearStarted (int)
- insuranceNumber (String)

Save this class in a file called **Employee.java**.

Your classes should have the necessary **constructors** and **getter/setter methods**.

Create another class called **TestEmployee** containing the main() method to fully test the classes.

Code:

Person.java:

```
Person.java > Person
1  class Person {
2
3      private String name;
4
5      // Constructor
6      Person(String name) {
7          this.name = name;
8      }
9
10     // Getter
11     public String getName() {
12         return name;
13     }
14
15     // Setter
16     public void setName(String name) {
17         this.name = name;
18     }
19 }
```

Employee.java:

```
class Employee extends Person {  
    private double annualSalary;  
    private int yearStarted;  
    private String insuranceNumber;  
  
    // Constructor  
    Employee(String name, double annualSalary,  
        int yearStarted, String insuranceNumber) {  
  
        super(name);  
  
        this.annualSalary = annualSalary;  
        this.yearStarted = yearStarted;  
        this.insuranceNumber = insuranceNumber;  
    }  
  
    // Getters  
    public double getAnnualSalary() {  
        return annualSalary;  
    }  
  
    public int getYearStarted() {  
        return yearStarted;  
    }  
  
    public String getInsuranceNumber() {  
        return insuranceNumber;  
    }  
}
```

```
// Setters

public void setAnnualSalary(double annualSalary) {

    this.annualSalary = annualSalary;

}

public void setYearStarted(int yearStarted) {

    this.yearStarted = yearStarted;

}

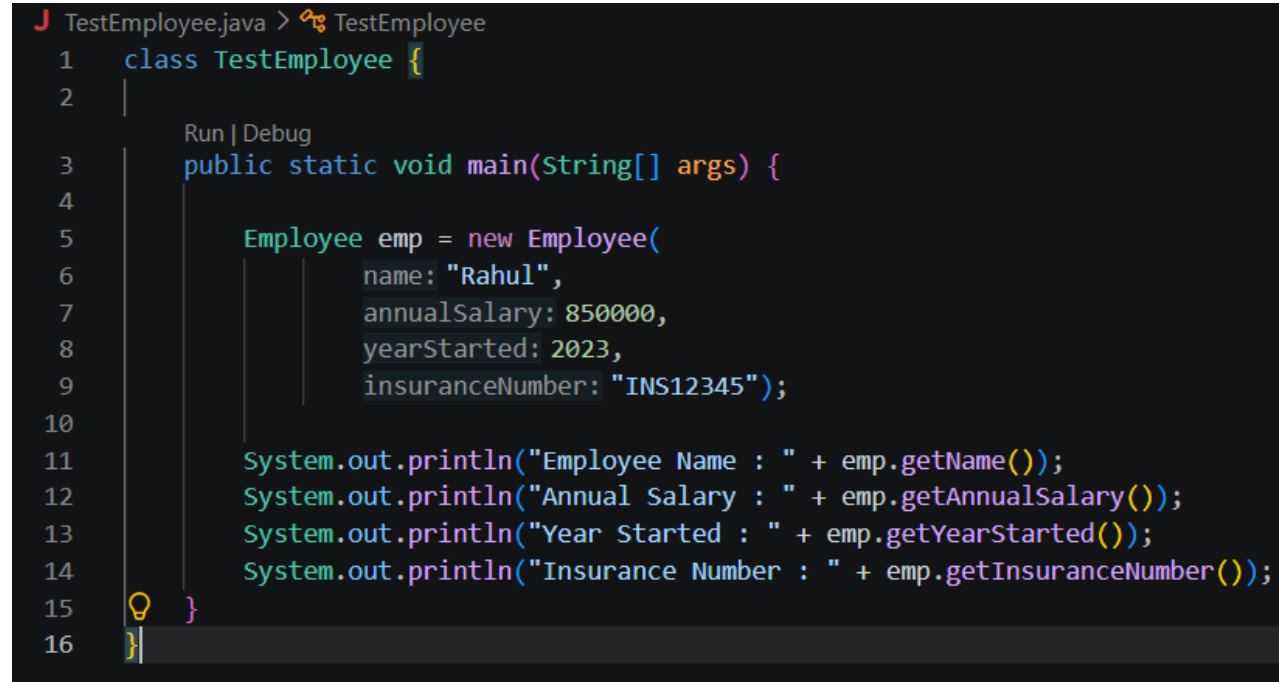
public void setInsuranceNumber(String insuranceNumber) {

    this.insuranceNumber = insuranceNumber;

}

}
```

TestEmployee.java:



```
J TestEmployee.java > TestEmployee
1  class TestEmployee {
2
3      Run | Debug
4      public static void main(String[] args) {
5
6          Employee emp = new Employee(
7              name: "Rahul",
8              annualSalary: 850000,
9              yearStarted: 2023,
10             insuranceNumber: "INS12345");
11
12             System.out.println("Employee Name : " + emp.getName());
13             System.out.println("Annual Salary : " + emp.getAnnualSalary());
14             System.out.println("Year Started : " + emp.getYearStarted());
15             System.out.println("Insurance Number : " + emp.getInsuranceNumber());
16         }
17     }
```

Output:

```
Insurance Number : INS12345
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java> javac Person.java Employee.java TestEmployee.java
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java> java TestEmployee
Employee Name : Rahul
Annual Salary : 850000.0
Year Started : 2023
Insurance Number : INS12345
```

Overriding & Polymorphism

Q40. Create a base class **Fruit** with the following attributes: name, taste, size. Create a method called eat() that displays the **name** of the fruit and its **taste**.

Create two child classes, **Apple** and **Orange**, that inherit from the Fruit class and **override** the eat() method to represent the taste of each fruit.

Code:

Fruit.java:

```
J Fruit.java > Fruit
1  class Fruit {
2
3      protected String name;
4      protected String taste;
5      protected String size;
6
7      // Parameterized Constructor
8      Fruit(String name, String taste, String size) {
9          this.name = name;
10         this.taste = taste;
11         this.size = size;
12     }
13
14     // Method
15     void eat() {
16         System.out.println("Fruit : " + name);
17         System.out.println("Taste : " + taste);
18     }
19 }
```

Apple.java:

```
J Apple.java > Apple
1  class Apple extends Fruit {
2
3      Apple() {
4          super(name: "Apple", taste: "Sweet", size: "Medium");
5      }
6
7      @Override
8      void eat() {
9          System.out.println("Fruit : " + name);
10         System.out.println("Taste : " + taste);
11     }
12 }
```

Orange.java:

```
J Orange.java > 🚀 Orange
1  class Orange extends Fruit {
2
3      Orange() {
4          super(name: "Orange", taste: "Sour", size: "Medium");
5      }
6
7      @Override
8      void eat() {
9          System.out.println("Fruit : " + name);
10         System.out.println("Taste : " + taste);
11     }
12 }
```

TestFruit.java:

```
J TestFruit.java > 🚀 TestFruit
1  class TestFruit {
2
3      Run | Debug
4      public static void main(String[] args) {
5
6          Apple apple = new Apple();
7          Orange orange = new Orange();
8
9          apple.eat();
10
11         System.out.println();
12
13         orange.eat();
14     }
15 }
```

Output:

```
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java> javac Fruit.java Apple.java Orange.java TestFruit.java
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java> java TestFruit
Fruit : Apple
Taste : Sweet

Fruit : Orange
Taste : Sour
```


Q41. Write a Java program to create a class named **Shape**. It should contain two methods:

- draw() – prints "**Drawing Shape**"
- erase() – prints "**Erasing Shape**"

Create three subclasses:

- Circle
- Triangle
- Square

Each subclass should **override** the parent class methods draw() and erase().

The methods should display the following messages:

- Circle
 - draw() → "**Drawing Circle**"
 - erase() → "**Erasing Circle**"
- Triangle
 - draw() → "**Drawing Triangle**"
 - erase() → "**Erasing Triangle**"
- Square
 - draw() → "**Drawing Square**"
 - erase() → "**Erasing Square**"

Create objects using **polymorphism** as follows:

Shape c = new Circle();

Shape t = new Triangle();

Shape s = new Square();

Call the draw() and erase() methods using each object.

Code:**Shape.java:**

```
J Shape.java > Shape
1  class Shape {
2
3      void draw() {
4          System.out.println(x: "Drawing Shape");
5      }
6
7      void erase() {
8          System.out.println(x: "Erasing Shape");
9      }
10 }
```

Circle.java:

```
J Circle.java > Circle
1  class Circle extends Shape {
2
3      @Override
4      void draw() {
5          System.out.println(x: "Drawing Circle");
6      }
7
8      @Override
9      void erase() {
10         System.out.println(x: "Erasing Circle");
11     }
12 }
```

Triangle.java:

```
J Triangle.java > Triangle
1  class Triangle extends Shape {
2
3      @Override
4      void draw() {
5          System.out.println(x: "Drawing Triangle");
6      }
7
8      @Override
9      void erase() {
10         System.out.println(x: "Erasing Triangle");
11     }
12 }
```

Square.java:

```
J Square.java >  Square
1  class Square extends Shape {
2
3      @Override
4      void draw() {
5          System.out.println(x: "Drawing Square");
6      }
7
8      @Override
9      void erase() {
10         System.out.println(x: "Erasing Square");
11     }
12 }
```

TestShape.java:

```
J TestShape.java >  TestShape
1  class TestShape {
2
3      Run | Debug
4      public static void main(String[] args) {
5
6          Shape c = new Circle();
7          Shape t = new Triangle();
8          Shape s = new Square();
9
10         c.draw();
11         c.erase();
12
13         System.out.println();
14
15         t.draw();
16         t.erase();
17
18         System.out.println();
19
20         s.draw();
21         s.erase();
22     }
23 }
```

Output:

```
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java> javac Shape.java Circle.java Triangle.java Square.java TestShape.java

PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java> java TestShape
Drawing Circle
Erasing Circle

Drawing Triangle
Erasing Triangle

Drawing Square
Erasing Square
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java> |
```

String & StringBuffer

Q42. Write a Java program to check whether a given **String** is a **Palindrome** or not.

```
J StringPalindrome.java > StringPalindrome
1  import java.util.Scanner;
2
3  class StringPalindrome {
    Run | Debug
4      public static void main(String[] args) {
5
6          Scanner sc = new Scanner(System.in);
7
8          System.out.print(s: "Enter a string: ");
9          String str = sc.nextLine();
10
11         String reverse = "";
12
13         for (int i = str.length() - 1; i >= 0; i--) {
14             reverse = reverse + str.charAt(i);
15         }
16
17         if (str.equalsIgnoreCase(reverse)) {
18             System.out.println(str + " is a Palindrome");
19         } else {
20             System.out.println(str + " is not a Palindrome");
21         }
22
23         sc.close();
24     }
25 }
```

OUTPUT TERMINAL

```
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java> javac StringPalindrome.java
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java> java StringPalindrome
Enter a string: MADAM
MADAM is a Palindrome
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java> java StringPalindrome
Enter a string: Hello
Hello is not a Palindrome
```

Q43. Write a Java program to **concatenate two strings** and return the result in **lowercase**.

Note: If the concatenation creates a **double character** (the last character of the first string is the same as the first character of the second string), then **one of the duplicate characters should be omitted**.

```
ConcatenateStrings.java > ConcatenateStrings
1  import java.util.Scanner;
2
3  class ConcatenateStrings {
4      public static void main(String[] args) {
5
6          Scanner sc = new Scanner(System.in);
7
8          System.out.print(s: "Enter first string: ");
9          String str1 = sc.nextLine();
10
11         System.out.print(s: "Enter second string: ");
12         String str2 = sc.nextLine();
13
14         String result;
15
16         if (str1.charAt(str1.length() - 1) == str2.charAt(index: 0)) {
17             result = str1 + str2.substring(beginIndex: 1);
18         } else {
19             result = str1 + str2;
20         }
21
22         System.out.println(result.toLowerCase());
23
24         sc.close();
25     }
26 }
```

OUTPUT TERMINAL

```
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java> javac ConcatenateStrings.java
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java> java ConcatenateStrings
Enter first string: Indranil
Enter second string: Mota
indranilmota
```

Q44. Given a string, return a new string made of 'n' copies of the first 2 characters of the original string, where 'n' is the length of the string.

```
J StringCopies.java > StringCopies
1  import java.util.Scanner;
2
3  class StringCopies {
4      public static void main(String[] args) {
5
6          Scanner sc = new Scanner(System.in);
7
8          System.out.print(s: "Enter a string: ");
9          String str = sc.nextLine();
10
11         String firstTwo = str.substring(beginIndex: 0, endIndex: 2);
12
13         for (int i = 0; i < str.length(); i++) {
14             System.out.print(firstTwo);
15         }
16
17         sc.close();
18     }
19 }
```

OUTPUT TERMINAL

```
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java> javac StringCopies.java
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java> java StringCopies
Enter a string: Hello
HeHeHeHeHe
```

Q45. Write a Java program that returns the **first half of the string**, if the **length of the string is even**. It should return **"null"** for an **odd-length** string.

```
J FirstHalfString.java > FirstHalfString
1  import java.util.Scanner;
2
3  class FirstHalfString {
    Run | Debug
4      public static void main(String[] args) {
5
6          Scanner sc = new Scanner(System.in);
7
8          System.out.print(s: "Enter a string: ");
9          String str = sc.nextLine();
10
11         if (str.length() % 2 == 0) {
12             System.out.println(str.substring(beginIndex: 0, str.length() / 2));
13         } else {
14             System.out.println(x: "null");
15         }
16
17         sc.close();
18     }
19 }
```

OUTPUT TERMINAL

```
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java> javac FirstHalfString.java
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java> java FirstHalfString
Enter a string: TomCat
Tom
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java> java FirstHalfString
Enter a string: Welcome
null
```


Q46. Write a Java program that accepts a **string** and returns a **new string without the first and last character** of the input string.

```
RemoveFirstLast.java > RemoveFirstLast
1  import java.util.Scanner;
2
3  class RemoveFirstLast {
4      public static void main(String[] args) {
5
6          Scanner sc = new Scanner(System.in);
7
8          System.out.print(s: "Enter a string: ");
9          String str = sc.nextLine();
10
11         System.out.println(str.substring(beginIndex: 1, str.length() - 1));
12
13         sc.close();
14     }
15 }
```

OUTPUT TERMINAL

```
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java> javac RemoveFirstLast.java
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java> java RemoveFirstLast
Enter a string: Wipro
ipr
```

Q47. Given two strings, a and b, return a new string of the form:

short + long + short

where the **shorter string** is placed on the outside and the **longer string** is placed in the middle.

Note:

- The two strings will **not** be of the same length.
- The strings may be empty (length = 0).

```
J ShortLongShort.java > ShortLongShort
1  import java.util.Scanner;
2
3  class ShortLongShort {
4      public static void main(String[] args) {
5
6          Scanner sc = new Scanner(System.in);
7
8          System.out.print(s: "Enter first string: ");
9          String a = sc.nextLine();
10
11         System.out.print(s: "Enter second string: ");
12         String b = sc.nextLine();
13
14         if (a.length() < b.length()) {
15             System.out.println(a + b + a);
16         } else {
17             System.out.println(b + a + b);
18         }
19
20         sc.close();
21     }
22 }
```

OUTPUT TERMINAL

```
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java> javac ShortLongShort.java
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java> java ShortLongShort
Enter first string: java
Enter second string: c
cjavac
```

Q48. Given a string, if the **first** or **last** character is 'x', return the string **without those 'x' characters**. Otherwise, return the string unchanged.

```
J RemoveX.java > RemoveX
1  import java.util.Scanner;
2
3  class RemoveX {
4      public static void main(String[] args) {
5
6          Scanner sc = new Scanner(System.in);
7
8          System.out.print(s: "Enter a string: ");
9          String str = sc.nextLine();
10
11         if (str.startsWith(prefix: "x")) {
12             str = str.substring(beginIndex: 1);
13         }
14
15         if (str.endsWith(suffix: "x")) {
16             str = str.substring(beginIndex: 0, str.length() - 1);
17         }
18
19         System.out.println(str);
20
21         sc.close();
22     }
23 }
```

OUTPUT TERMINAL

```
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java> javac RemoveX.java
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java> java RemoveX
Enter a string: xhix
hi
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java> java RemoveX
Enter a string: India
India
```

Q49. Write a Java program that accepts a **string** (which contains * in it). The program should return a **new string** in which the following characters are removed:

- The * character.
- The character immediately to the **left** of *.
- The character immediately to the **right** of *.

```
J RemoveStar.java > RemoveStar
1  import java.util.Scanner;
2
3  class RemoveStar {
4      public static void main(String[] args) {
5
6          Scanner sc = new Scanner(System.in);
7
8          System.out.print(s: "Enter a string: ");
9          String str = sc.nextLine();
10
11         int index = str.indexOf(ch: '*');
12
13         String result = str.substring(beginIndex: 0, index - 1)
14             + str.substring(index + 2);
15
16         System.out.println(result);
17
18         sc.close();
19     }
20 }
```

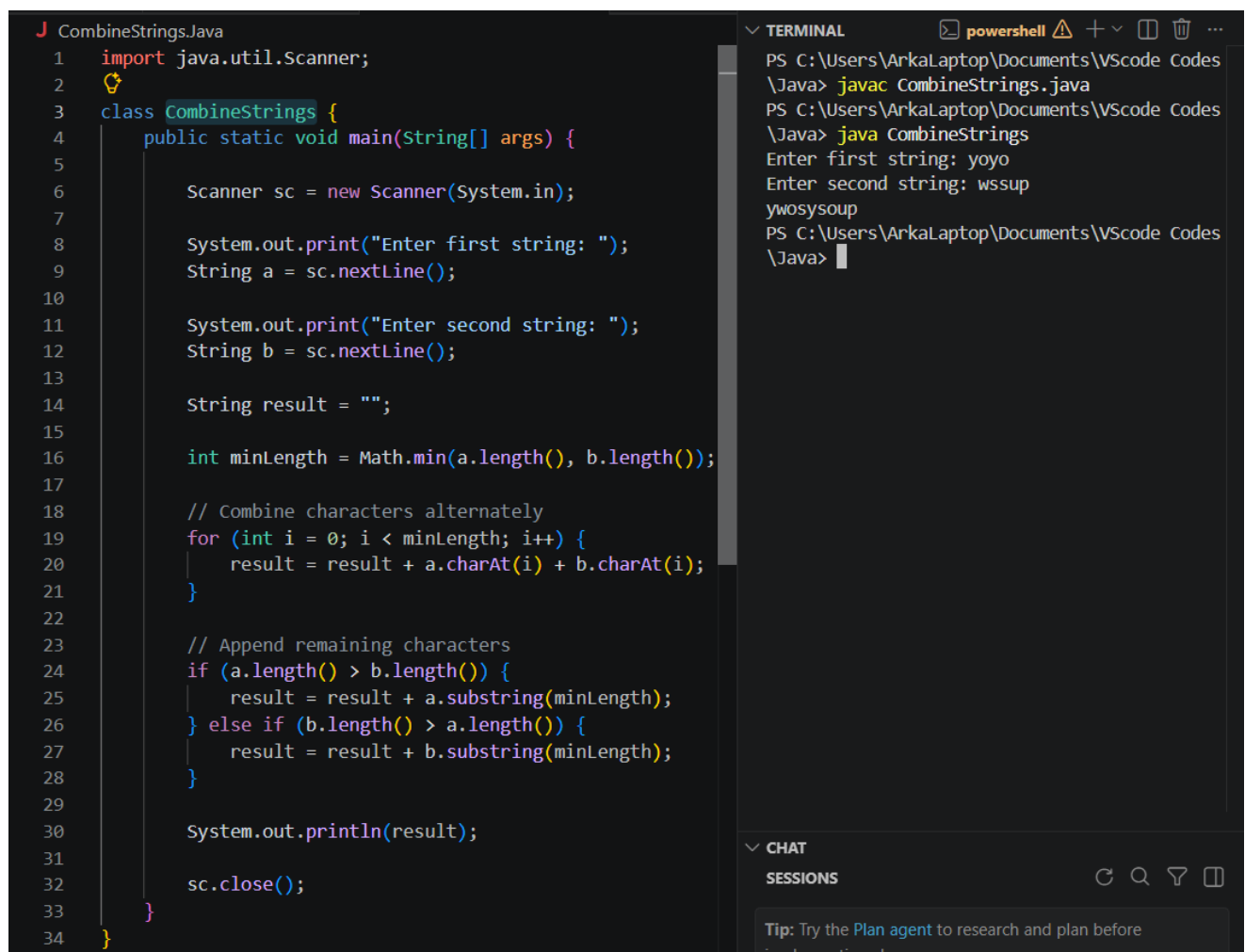
OUTPUT TERMINAL

```
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java> javac RemoveStar.java
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java> java RemoveStar
Enter a string: ab*cd
ad
```

Q50. Given two strings, a and b, print a new string by combining them as follows:

- First character of a
- First character of b
- Second character of a
- Second character of b
- and so on...

If one string is longer than the other, append the remaining characters at the end of the result.



```
J CombineStrings.java
1  import java.util.Scanner;
2
3  class CombineStrings {
4      public static void main(String[] args) {
5
6          Scanner sc = new Scanner(System.in);
7
8          System.out.print("Enter first string: ");
9          String a = sc.nextLine();
10
11         System.out.print("Enter second string: ");
12         String b = sc.nextLine();
13
14         String result = "";
15
16         int minLength = Math.min(a.length(), b.length());
17
18         // Combine characters alternately
19         for (int i = 0; i < minLength; i++) {
20             result = result + a.charAt(i) + b.charAt(i);
21         }
22
23         // Append remaining characters
24         if (a.length() > b.length()) {
25             result = result + a.substring(minLength);
26         } else if (b.length() > a.length()) {
27             result = result + b.substring(minLength);
28         }
29
30         System.out.println(result);
31
32         sc.close();
33     }
34 }
```

TERMINAL

```
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java> javac CombineStrings.java
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java> java CombineStrings
Enter first string: yoyo
Enter second string: wssup
ywosysoup
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java>
```

CHAT

SESSIONS

Tip: Try the [Plan agent](#) to research and plan before implementing changes

Q51. Given a string and an integer n, print a new string made of **n repetitions of the last n characters** of the string.

You may assume that n is between 0 and the length of the string, inclusive.

```
J RepeatLastN.java > RepeatLastN
1  import java.util.Scanner;
2
3  class RepeatLastN {
4      public static void main(String[] args) {
5
6          Scanner sc = new Scanner(System.in);
7
8          System.out.print(s: "Enter a string: ");
9          String str = sc.nextLine();
10
11         System.out.print(s: "Enter n: ");
12         int n = sc.nextInt();
13
14         // Get the last n characters
15         String lastPart = str.substring(str.length() - n);
16
17         // Print lastPart n times
18         for (int i = 0; i < n; i++) {
19             System.out.print(lastPart);
20         }
21
22         sc.close();
23     }
24 }
```

OUTPUT TERMINAL

```
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java> javac RepeatLastN.java
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java> java RepeatLastN
Enter a string: Wipro
Enter n: 3
propropro
```

StringJoiner Class

Q52. Given an ArrayList containing **n names**, use **StringJoiner** to construct a new sequence of names separated by , (**comma**) and enclosed in { } brackets.

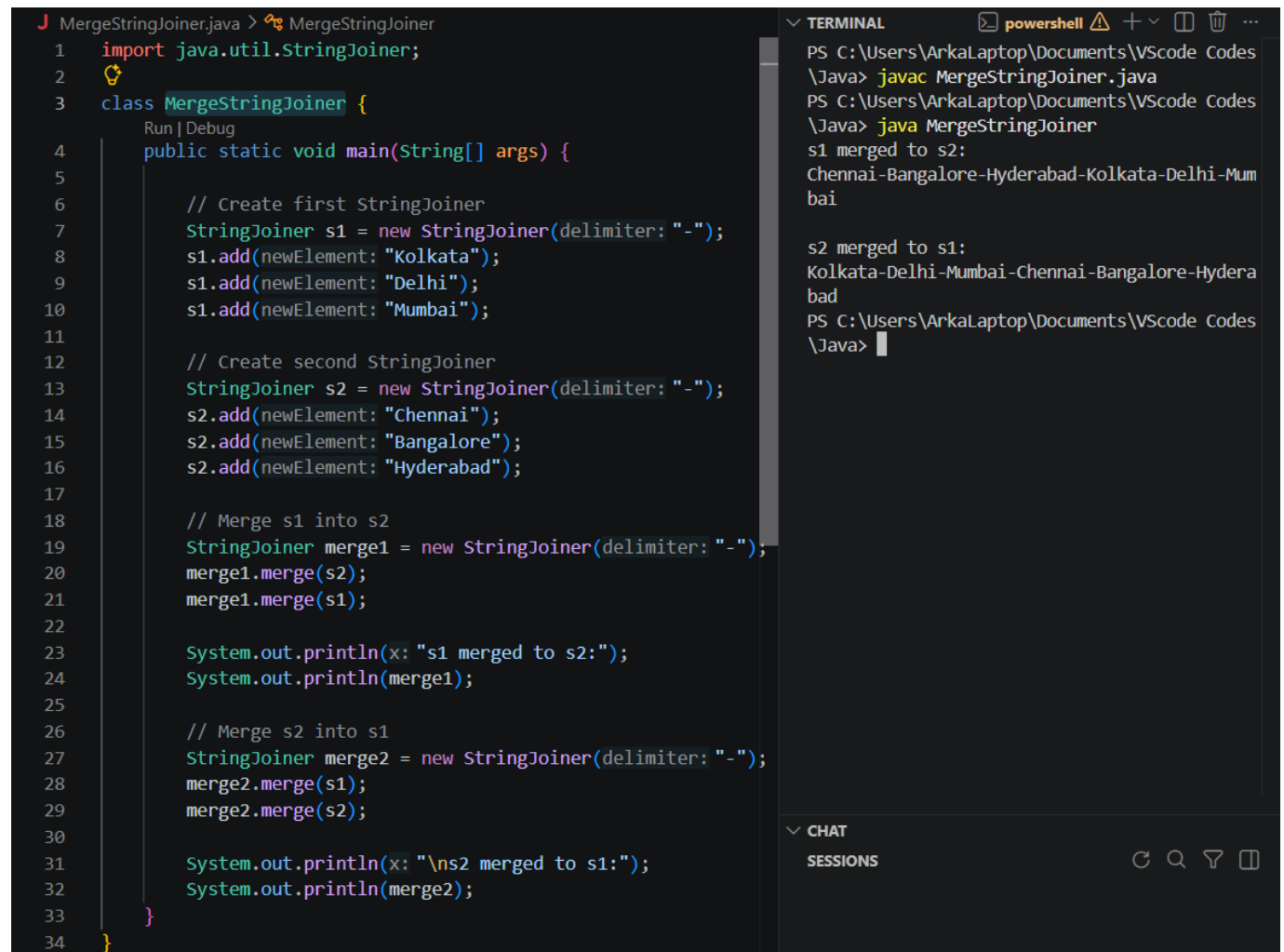
```
J JoinNames.java > JoinNames > main(String[])
1  import java.util.ArrayList;
2  import java.util.StringJoiner;
3
4  class JoinNames {
5      Run | Debug
6      public static void main(String[] args) {
7
8          // Create ArrayList
9          ArrayList<String> names = new ArrayList<>();
10
11         // Add names
12         names.add(e: "Arka");
13         names.add(e: "Sayasn");
14         names.add(e: "Sabuj");
15         names.add(e: "Indranil");
16
17         // Create StringJoiner
18         StringJoiner sj = new StringJoiner(delimiter: ",", prefix: "{", suffix: "}");
19
20         // Add names to StringJoiner
21         for (String name : names) {
22             sj.add(name);
23         }
24         // Print result
25         System.out.println(sj);
26     }
27 }
```

OUTPUT TERMINAL powershell

```
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java> javac JoinNames.java
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java> java JoinNames
{Arka, Saysn, Sabuj, Indranil}
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java> 
```

Q53. Given two `StringJoiner` objects `s1` and `s2` containing city names separated by **hyphens (-)**, display the output for the following cases:

1. Merge `s1` into `s2`.
2. Merge `s2` into `s1`.



```
J MergeStringJoiner.java > MergeStringJoiner
1  import java.util.StringJoiner;
2
3  class MergeStringJoiner {
    Run | Debug
4      public static void main(String[] args) {
5
6          // Create first StringJoiner
7          StringJoiner s1 = new StringJoiner(delimiter: "-");
8          s1.add(newElement: "Kolkata");
9          s1.add(newElement: "Delhi");
10         s1.add(newElement: "Mumbai");
11
12         // Create second StringJoiner
13         StringJoiner s2 = new StringJoiner(delimiter: "-");
14         s2.add(newElement: "Chennai");
15         s2.add(newElement: "Bangalore");
16         s2.add(newElement: "Hyderabad");
17
18         // Merge s1 into s2
19         StringJoiner merge1 = new StringJoiner(delimiter: "-");
20         merge1.merge(s2);
21         merge1.merge(s1);
22
23         System.out.println(x: "s1 merged to s2:");
24         System.out.println(merge1);
25
26         // Merge s2 into s1
27         StringJoiner merge2 = new StringJoiner(delimiter: "-");
28         merge2.merge(s1);
29         merge2.merge(s2);
30
31         System.out.println(x: "\ns2 merged to s1:");
32         System.out.println(merge2);
33     }
34 }
```

TERMINAL powershell

```
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java> javac MergeStringJoiner.java
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java> java MergeStringJoiner
s1 merged to s2:
Chennai-Bangalore-Hyderabad-Kolkata-Delhi-Mumbai

s2 merged to s1:
Kolkata-Delhi-Mumbai-Chennai-Bangalore-Hyderabad
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java>
```

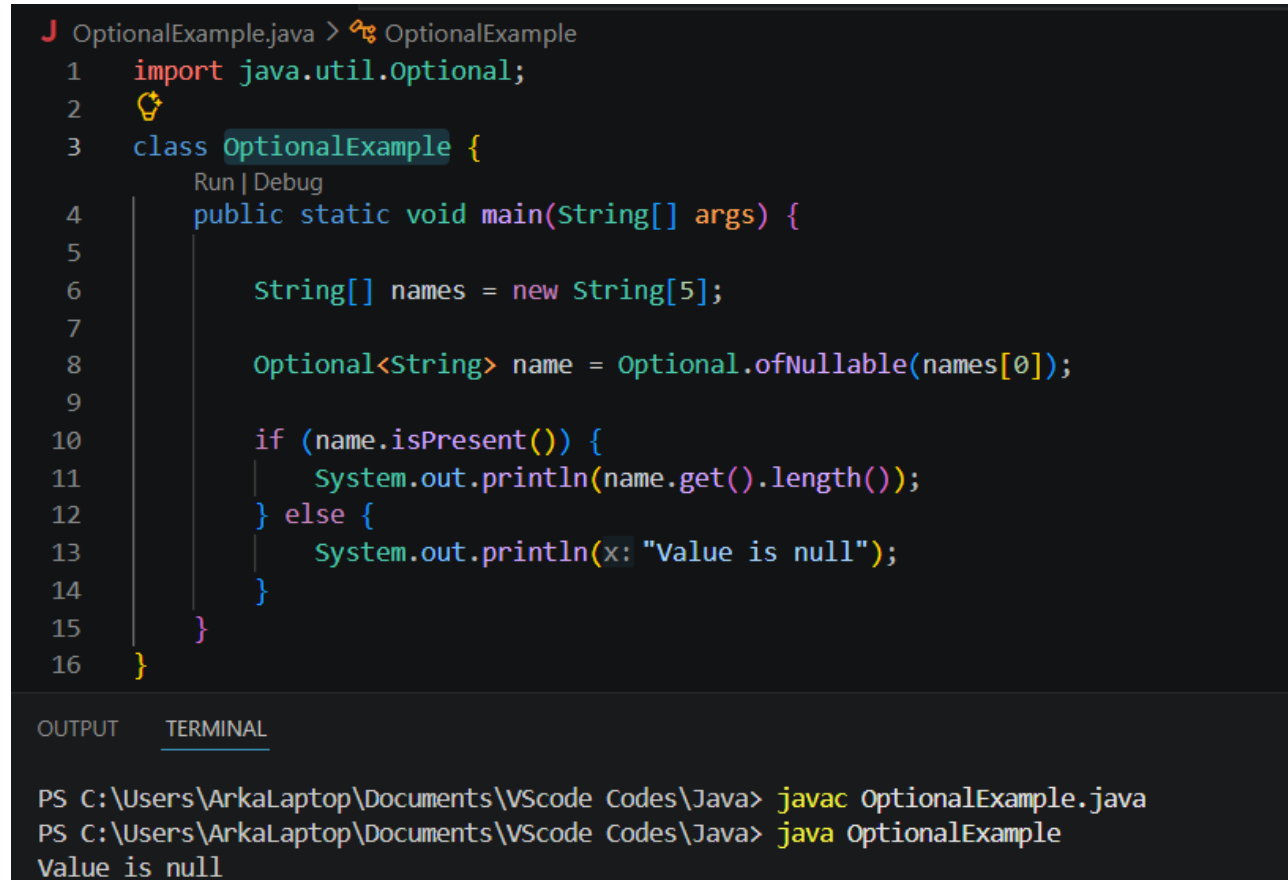
CHAT

SESSIONS

Optional Class

Q54. Use the **Optional** class to avoid a *NullPointerException* from the following code:

```
String names[] = new String[5];  
System.out.print(names[0].length());
```



```
J OptionalExample.java > OptionalExample  
1  import java.util.Optional;  
2  
3  class OptionalExample {  
4      Run | Debug  
5      public static void main(String[] args) {  
6  
7          String[] names = new String[5];  
8  
9          Optional<String> name = Optional.ofNullable(names[0]);  
10  
11         if (name.isPresent()) {  
12             System.out.println(name.get().length());  
13         } else {  
14             System.out.println(x: "Value is null");  
15         }  
16     }  
}
```

OUTPUT TERMINAL

```
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java> javac OptionalExample.java  
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java> java OptionalExample  
Value is null
```

Q55. Given a String variable address, print the address if it is **not null**; otherwise, print the default address as **"India"**.

```
J AddressExample.java > AddressExample
2
3 class AddressExample {
4     public static void main(String[] args) {
5
6         String address = null;
7
8         String result = Optional.ofNullable(address)
9                               .orElse("India");
10
11        System.out.println(result);
12    }
13 }
```

OUTPUT TERMINAL

```
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java> javac AddressExample.java
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java> java AddressExample
India
```

Q56. Given an Employee object that may be initialized to null, use the **Optional** class to check whether it is null and throw a **user-defined exception** named InvalidEmployeeException if it is null.

Code:

```
import java.util.Optional;

// User-defined Exception

class InvalidEmployeeException extends Exception {

    InvalidEmployeeException(String message) {
        super(message);
    }
}

// Employee Class

class Employee {

    private int id;
    private String name;

    Employee(int id, String name) {
        this.id = id;
        this.name = name;
    }

    void display() {
        System.out.println("Employee ID : " + id);
        System.out.println("Employee Name : " + name);
    }
}
```

```
}

class TestEmployee2 {

    public static void main(String[] args) {

        Employee emp = null;

        try {

            Optional<Employee> employee = Optional.ofNullable(emp);

            if (employee.isPresent()) {
                employee.get().display();
            } else {
                throw new InvalidEmployeeException("Invalid Employee");
            }

        } catch (InvalidEmployeeException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Output:

```
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java> javac TestEmployee2.java
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java> java TestEmployee2
Invalid Employee
```

Module Mini Project

Video Rental Inventory System

The goal of this project is to design and implement a simple inventory control system for a small video rental store.

The following are the various classes that are to be implemented.

1. Video

Member variables

- String videoName
- boolean checkout
- int rating

Member functions

- String getName();
- void doCheckout();
- void doReturn();
- void receiveRating(int rating);
- int getRating();
- boolean getCheckout();

Constructor

- Video(String name)

2. VideoStore

Member variables

- Video[] store;

Member functions

- void addVideo(String name);
- void doCheckout(String name);
- void doReturn(String name);
- void receiveRating(String name, int rating);
- void listInventory();

3. VideoLaucher

Contains the main method to test the program

Sample Output:

D:\Batches\Milestone1> java VideoLauncher

MAIN MENU

=====

- 1.Add Videos:
- 2.Check Out Video :
- 3.Return Video :
- 4.Receive Rating :
- 5.List Inventory :
- 6.Exit :

Enter your choice (1..6): 1

Enter the name of the video you want to add: Matrix
Video "Matrix" added successfully.

MAIN MENU

=====

- 1.Add Videos:
- 2.Check Out Video :
- 3.Return Video :
- 4.Receive Rating :
- 5.List Inventory :
- 6.Exit :

Enter your choice (1..6): 4

Enter the name of the video you want to Rate: Matrix
Enter the rating for this video: 9
Rating "9" has been mapped to the Video "Matrix".

MAIN MENU

=====

- 1.Add Videos:
- 2.Check Out Video :
- 3.Return Video :
- 4.Receive Rating :
- 5.List Inventory :
- 6.Exit :

Enter your choice (1..6): 2

Enter the name of the video you want to check out: Matrix
Video "Matrix" checked out successfully.

MAIN MENU

=====

1.Add Videos:

2.Check Out Video :

3.Return Video :

4.Receive Rating :

5.List Inventory :

6.Exit :

Enter your choice (1..6): 5

```
-----
Video Name      |      Checkout Status |      Rating
Matrix          |      true             |      9
-----
```

MAIN MENU

=====

1.Add Videos:

2.Check Out Video :

3.Return Video :

4.Receive Rating :

5.List Inventory :

6.Exit :

Enter your choice (1..6): 3

Enter the name of the video you want to Return: Matrix

Video "Matrix" returned successfully.

MAIN MENU

=====

1.Add Videos:

2.Check Out Video :

3.Return Video :

4.Receive Rating :

5.List Inventory :

6.Exit :

Enter your choice (1..6): 5

```
-----
Video Name      |      Checkout Status |      Rating
Matrix          |      false           |      9
-----
```

Code:**Video.java:**

```
class Video {  
    // Member variables  
  
    private String videoName;  
    private boolean checkout;  
    private int rating;  
  
    // Constructor  
    Video(String name) {  
        videoName = name;  
        checkout = false;  
        rating = 0;  
    }  
  
    // Returns video name  
    public String getName() {  
        return videoName;  
    }  
  
    // Checkout the video  
    public void doCheckout() {  
        checkout = true;  
    }  
  
    // Return the video  
    public void doReturn() {  
        checkout = false;  
    }  
}
```



```
// Receive rating
public void receiveRating(int rating) {
    this.rating = rating;
}

// Return rating
public int getRating() {
    return rating;
}

// Return checkout status
public boolean getCheckout() {
    return checkout;
}
}
```

VideoStore.java:

```
class VideoStore {
    // Array to store videos
    private Video[] store;

    // Keeps track of number of videos
    private int count;

    // Constructor
    VideoStore() {
        store = new Video[10]; // Can store up to 10 videos
        count = 0;
    }
}
```

```
}
```

```
// Add a new video
```

```
public void addVideo(String name) {
```

```
    store[count] = new Video(name);
```

```
    count++;
```

```
    System.out.println("Video \"" + name + "\" added successfully.");
```

```
}
```

```
// Checkout a video
```

```
public void doCheckout(String name) {
```

```
    for (int i = 0; i < count; i++) {
```

```
        if (store[i].getName().equalsIgnoreCase(name)) {
```

```
            store[i].doCheckout();
```

```
            System.out.println("Video \"" + name + "\" checked out successfully.");
```

```
            return;
```

```
        }
```

```
    }
```

```
    System.out.println("Video not found.");
```

```
}
```

```
// Return a video
```

```
public void doReturn(String name) {

    for (int i = 0; i < count; i++) {

        if (store[i].getName().equalsIgnoreCase(name)) {

            store[i].doReturn();

            System.out.println("Video \"" + name + "\" returned successfully.");

            return;
        }
    }

    System.out.println("Video not found.");
}

// Receive rating
public void receiveRating(String name, int rating) {

    for (int i = 0; i < count; i++) {

        if (store[i].getName().equalsIgnoreCase(name)) {

            store[i].receiveRating(rating);

            System.out.println("Rating \"" + rating
                + "\" has been mapped to the Video \"" + name + "\".");

            return;
        }
    }
}
```

```
    }  
}  
  
    System.out.println("Video not found.");  
}  
  
// Display inventory  
public void listInventory() {  
  
    System.out.println("-----");  
    System.out.printf("%-20s %-20s %-10s%n",  
        "Video Name", "Checkout Status", "Rating");  
    System.out.println("-----");  
  
    for (int i = 0; i < count; i++) {  
  
        System.out.printf("%-20s %-20s %-10d%n",  
            store[i].getName(),  
            store[i].getCheckout(),  
            store[i].getRating());  
    }  
  
    System.out.println("-----");  
}  
}
```

VideoLauncher.java:

```
import java.util.Scanner;

class VideoLauncher {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        VideoStore store = new VideoStore();

        int choice;

        do {

            System.out.println("\nMAIN MENU");
            System.out.println("=====");
            System.out.println("1. Add Video");
            System.out.println("2. Check Out Video");
            System.out.println("3. Return Video");
            System.out.println("4. Receive Rating");
            System.out.println("5. List Inventory");
            System.out.println("6. Exit");

            System.out.print("Enter your choice (1-6): ");
            choice = sc.nextInt();
            sc.nextLine();

            switch (choice) {
```

case 1:

```
System.out.print("Enter video name: ");
```

```
String addName = sc.nextLine();
```

```
store.addVideo(addName);
```

```
break;
```

case 2:

```
System.out.print("Enter video name: ");
```

```
String checkoutName = sc.nextLine();
```

```
store.doCheckout(checkoutName);
```

```
break;
```

case 3:

```
System.out.print("Enter video name: ");
```

```
String returnName = sc.nextLine();
```

```
store.doReturn(returnName);
```

```
break;
```

case 4:

```
System.out.print("Enter video name: ");
```

```
String ratingName = sc.nextLine();
```

```
System.out.print("Enter rating: ");
```

```
        int rating = sc.nextInt();

        store.receiveRating(ratingName, rating);

        break;

    case 5:

        store.listInventory();

        break;

    case 6:

        System.out.println("Exiting... Thanks for using the application.");

        break;

    default:

        System.out.println("Invalid Choice.");

    }

} while (choice != 6);

sc.close();
}
}
```

Output:

```
✓ TERMINAL java ⚠ + - □ 🗑 ...

PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java Projects\VideoStore>
javac Video.java
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java Projects\VideoStore>
javac VideoStore.java
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java Projects\VideoStore>
Javac VideoLauncher.java
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java Projects\VideoStore>
java VideoLauncher

MAIN MENU
=====
1. Add Video
2. Check Out Video
3. Return Video
4. Receive Rating
5. List Inventory
6. Exit
Enter your choice (1-6): 1
Enter video name: SpiderMan
Video "SpiderMan" added successfully.

MAIN MENU
=====
1. Add Video
2. Check Out Video
3. Return Video
4. Receive Rating
5. List Inventory
6. Exit
Enter your choice (1-6): 1
Enter video name: IronMan

Video "IronMan" added successfully.

MAIN MENU
=====
1. Add Video
2. Check Out Video
3. Return Video
4. Receive Rating
5. List Inventory
6. Exit
Enter your choice (1-6): 1
Enter video name: Punisher
Video "Punisher" added successfully.
```



```
✓ TERMINAL java ⚠ + ▢ 🗑 ...

MAIN MENU
=====
1. Add Video
2. Check Out Video
3. Return Video
4. Receive Rating
5. List Inventory
6. Exit
Enter your choice (1-6): 1
Enter video name: Punisher
Video "Punisher" added successfully.

MAIN MENU
=====
1. Add Video
2. Check Out Video
3. Return Video
4. Receive Rating
5. List Inventory
6. Exit
Enter your choice (1-6): 5
-----
Video Name          Checkout Status      Rating
-----
SpiderMan           false                0
IronMan              false                0
Punisher             false                0
-----

MAIN MENU
=====
1. Add Video
2. Check Out Video
3. Return Video
4. Receive Rating
5. List Inventory
6. Exit
Enter your choice (1-6): 4
Enter video name: SpiderMan
Enter rating: 9
Rating "9" has been mapped to the Video "SpiderMan".
```

```
✓ TERMINAL java ⚠ + ▢ 🗑 ...

MAIN MENU
=====
1. Add Video
2. Check Out Video
3. Return Video
4. Receive Rating
5. List Inventory
6. Exit
Enter your choice (1-6): 4
Enter video name: IronMan
Enter rating: 10
Rating "10" has been mapped to the Video "IronMan".

MAIN MENU
=====
1. Add Video
2. Check Out Video
3. Return Video
4. Receive Rating
5. List Inventory
6. Exit
Enter your choice (1-6): 4
Enter video name: Punisher
Enter rating: 8
Rating "8" has been mapped to the Video "Punisher".
```

```
MAIN MENU
=====
1. Add Video
2. Check Out Video
3. Return Video
4. Receive Rating
5. List Inventory
6. Exit
Enter your choice (1-6): 5
-----
Video Name      Checkout Status    Rating
-----
SpiderMan       false              9
IronMan         false              10
Punisher        false              8
-----
```